

03 Embodied AI Applications in Vision Tracking

1. Brief Game Instructions

In this lesson, we will learn how to leverage a large model to analyze real-world scenes in real time and implement an interactive real-time detection activity. Note: The large language model used in this lesson is cloud-based. Before starting, please connect the robot to the internet using an Ethernet cable or switch it to local area network (LAN) mode.

Once the program starts running and WonderEcho Pro announces "I am ready."

Say the wake-up word (the wake-up word depends on the firmware flashed, and by default, the wake-up word for our factory firmware is "Hello, HiWonder") to activate WonderEcho Pro. It will respond with "I am here."

After activation, you can interact with MentorPi using voice commands. You can then control MentorPi with your voice. For example, say "**Track the purple ball.**" The terminal will display the recognized voice command, and the voice device will announce the generated response after processing, then execute the corresponding action.

2. Preparation

2.1 Verifying the WonderEcho Pro Firmware

By default, the wake-up word for the WonderEcho Pro factory firmware is "Hello, Hiwonder." If you wish to change the wake-up word to "MentorPi," please refer to the tutorial titled "03 Firmware Flashing Method" for

step-by-step instructions.

If you have previously flashed the device with different firmware—such as the version with the wake word “小幻小幻” or “mentorpi”—you must use the corresponding wake word to activate the device.

For example, if the “小幻小幻” firmware was installed, the correct wake word would be “小幻小幻”; if the “mentorpi” firmware was installed, the wake word would be “mentorpi.”

In the following tutorial, we will use the default factory wake word “Hello, Hiwonder” for demonstration purposes.

2.2 Obtaining and Configuring the Large Model API Key

By default, the program does not include the configuration of the Large AI Model-related API keys. Before activating features related to the Large AI Model, please refer to the section "02 Multimodal Large Model Applications > 01 Obtaining and Configuring Large Model API Keys" to configure the necessary keys. This step is mandatory and cannot be skipped, as it is crucial for the proper functioning and experience of the large model features.

2.3 Network Configuration

Note: The large language model used in this lesson is cloud-based. Before starting, please connect the robot to the internet using an Ethernet cable or switch it to local area network (LAN) mode.

The robot must be connected to the internet, either in STA (local network) mode or AP (direct connection) mode via Ethernet. For detailed instructions on network configuration, please refer to the course “20 Network Configuration

2.4 Turn Off Cooling Fan

- 1) Click  in the top-left corner to open the terminal.
- 2) Type the command and press Enter to turn off the Raspberry Pi’s cooling fan, minimizing noise interference during recording.

```
pinctrl FAN_PWM op dh
```



3. Starting and Stopping the Game

1. The input commands must strictly observe case sensitivity and spacing.

2. The robot must be connected to the internet, either in STA (local network) mode or AP (direct connection) mode via Ethernet.

- 1) Start the robot and follow the instructions in “8. Remote Tool Installation Connection → 8.4 Docker Container Introduction and Entry” to connect via the VNC remote desktop software. Then, switch to the ROS2 environment and select the appropriate language version as needed.
- 2) Double-click to launch the command bar. Enter the command and press Enter to disable the auto-start service.

```
~/stop_ros.sh
```

```
> ~/stop_ros.sh  
zsh: killed ~/stop_ros.sh
```

- 3) Enter the command to launch the corresponding game.

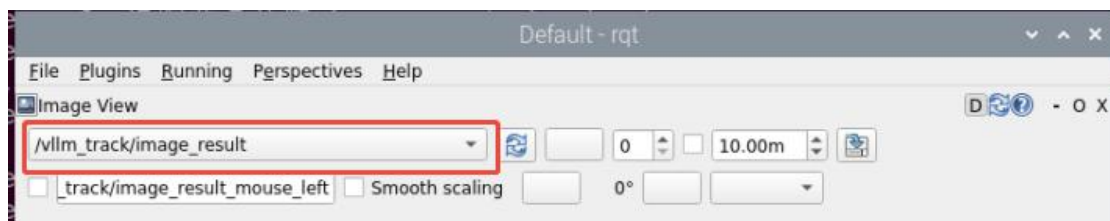
ros2 launch large_models vllm_track.launch.py

```
ros2 launch large_models vllm_track.launch.py
```

- 4) Open a new command-line terminal to execute the following command and access the live camera feed:

rqt

- 5) Choose the corresponding topic.



- 6) Once the command line outputs the message below and the device announces "I am ready," it indicates that the voice device has been initialized. You can now say the wake word, "Hello Hiwonder."

```
[agent_process-10] [INFO] [1742291623.374386995] [agent_process]: start
[tts_node-11] [INFO] [1742291624.176498371] [tts_node]: start
[vocal_detect-9] [INFO] [1742291624.184420787] [vocal_detect]: start
[ascamera_node-6] 2025-03-18 09:53:44[INFO] [CameraHp60c.cpp] [260] [startStreaming] start streaming
[agent_process-10] [INFO] [1742291624.996223930] [agent_process]: set model
[vllm_with_camera-12] [INFO] [1742291625.255926774] [vllm_with_camera]: start
```

- 7) When the following output appears in the command line, MentorPi will announce "I am Here," indicating that MentorPi has been successfully activated. At this point, the system will begin recording the user's command.

```
[vocal_detect-7] [INFO] [1741419606.799872983] [vocal_detect]: asr...
[vocal_detect-7] [INFO] [1741419606.805699617] [vocal_detect]: Recording.....
[vocal_detect-7] [INFO] [1741419608.378630341] [vocal_detect]:
```

- 8) You can freely phrase your commands to control the movement of MentorPi, such as saying, **"Track the purple ball."**

```
[vocal_detect-13] [INFO] [1747368528.955020449] [vocal_detect]: Done recording
[vocal_detect-13] [INFO] [1747368530.299871714] [vocal_detect]: asr time: 1.34
[vocal_detect-13] [INFO] [1747368530.956338694] [vocal_detect]: publish asr result:Track the purple ball
```

- 9) When the command line displays the output shown below, it indicates that MentorPi has printed the recognized speech.

```
[vocal_detect-13] [INFO] [1747368530.299871714] [vocal_detect]: asr time: 1.34
[vocal_detect-13] [INFO] [1747368530.956338694] [vocal_detect]: publish asr result:Track the purple ball
[vocal_detect-13]
```

- 10) When the command line shows the output below, it indicates that the cloud-based voice large model's speech recognition service has successfully processed the user's command audio. The recognition result will be displayed under "publish_asr_result."

```
[vocal_detect-13] [INFO] [1747368530.956338694] [vocal_detect]: publish asr result:Track the purple ball
[vocal_detect-13]
[agent_process-14] [INFO] [1747368530.959627633] [agent_process]: thinking...
```

- 11) When the command line shows the output below, it indicates that the cloud-based large language model has successfully processed the user's command, thought through the instruction, and provided a verbal response ("response"), as well as designed an action that aligns with the user's command meaning.

The response is automatically generated by the large model, ensuring the accuracy of the meaning, though the wording and structure of the reply may vary.

```
[agent_process-8] [INFO] [1741434900.673647362] [agent_process]: publish vllm result:{
[agent_process-8]   "object": "ball",
[agent_process-8]   "xyxy": [710, 672, 846, 850]
[agent_process-8] }
```

- 12) When the command line shows the output below, it indicates that one round of interaction has been completed. You can refer to Step 4) to reactivate the voice device using the wake-up word and begin a new round of interaction.

```
[agent_process-8]
[vllm_track-10] [INFO] [1741434900.699319302] [vllm_track]: box: [454, 322, 87, 86]
[vocal_detect-7] [INFO] [1741434900.715146762] [vocal_detect]: enable_wakeup
```

- 13) To exit this feature, simply press "Ctrl+C" in the terminal. If it doesn't close

on the first attempt, you may need to press it multiple times.

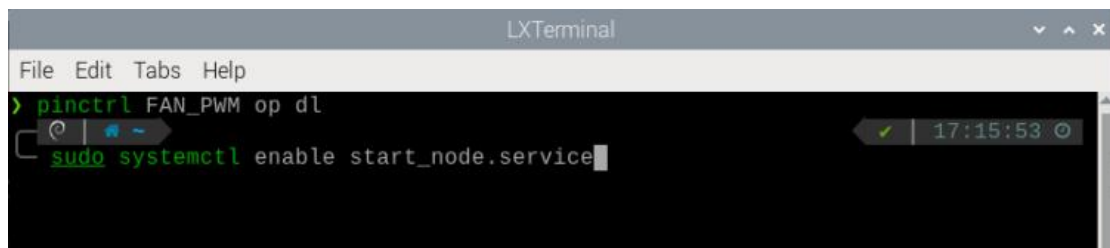
- 14) Type the command and press Enter to turn on the Raspberry Pi cooling fan.

```
pinctrl FAN_PWM op dl
```



- 15) Enter the command and press the Enter key to start the game:

```
sudo systemctl enable start_node.service
```



4. How It Works

Once the feature is activated, we can freely organize our commands, for example: **"Track the purple ball."** This will control the MentorPi robot car to automatically track the purple ball within its field of view, highlight the ball in the frame, and stop tracking when the ball is positioned at the center of the screen.

5. Brief Program Analysis

5.5 Analysis of launch File

The program is saved at:

/home/ubuntu/ros2_ws/src/large_models/launch/vllm_track.launch.py

◆ Library Files Import

```
1 import os
2 from ament_index_python.packages import get_package_share_directory
3
4 from launch_ros.actions import Node
5 from launch.substitutions import LaunchConfiguration
6 from launch import LaunchDescription, LaunchService
7 from launch.launch_description_sources import PythonLaunchDescriptionSource
8 from launch.actions import IncludeLaunchDescription, DeclareLaunchArgument, OpaqueFunction
```

① `ament_index_python.packages.get_package_share_directory`: Used to obtain the shared directory path of a ROS2 package.

② `launch_ros.actions.Node`: Used to define a ROS2 node

③ `launch.substitutions.LaunchConfiguration`: Used to retrieve parameter configurations from a Launch file.

④ `LaunchDescription, LaunchService`: Used to define and launch a Launch file.

⑤ `launch_description_sources PythonLaunchDescriptionSource`: Used to include other Launch files.

⑥ `launch.actions.IncludeLaunchDescription, DeclareLaunchArgument, OpaqueFunction`: Used to define actions and parameters within a Launch file.

◆ Define `launch_setup` Function


```

10 def launch_setup(context):
11     mode = LaunchConfiguration('mode', default=1)
12     mode_arg = DeclareLaunchArgument('mode', default_value=mode)
13
14     slam_package_path = get_package_share_directory('slam')
15     large_models_package_path = get_package_share_directory('large_models')
16
17     base_launch = IncludeLaunchDescription(
18         PythonLaunchDescriptionSource(
19             os.path.join(slam_package_path, 'launch/include/robot.launch.py')),
20         launch_arguments={
21             'sim': 'false',
22             'master_name': os.environ['MASTER'],
23             'robot_name': os.environ['HOST']
24         }.items(),
25     )
26
27     large_models_launch = IncludeLaunchDescription(
28         PythonLaunchDescriptionSource(
29             os.path.join(large_models_package_path, 'launch/start.launch.py')),
30         launch_arguments={'mode': mode}.items(),
31     )
32
33     vllm_track_node = Node(
34         package='large_models',
35         executable='vllm_track',
36         name='vllm_track',
37         output='screen',
38     )
39
40     # rqt
41     calibrate_rqt_reconfigure_node = Node(
42         package='rqt_reconfigure',
43         executable='rqt_reconfigure',
44         name='calibrate_rqt_reconfigure'
45     )

```

- ① `mode = LaunchConfiguration('mode', default=1)`: Defines a Launch parameter named `mode`, with a default value of 1.
- ② `sdk_launch`: Uses `IncludeLaunchDescription` to include the `MentorPi_sdk.launch.py` launch file from the `large_models` package, passing the `mode` parameter.
- ③ `large_models_launch`: Uses `IncludeLaunchDescription` to include the `start.launch.py` Launch file from the `large_models` package, passing the `mode` parameter.
- ④ `vllm_track_node`: Defines a ROS2 node from the `large_models` package that executes the `vllm_track` executable, and prints its output to the screen.

◆ **Define `generate_launch_description` function:**


```
54 def generate_launch_description():
55     return LaunchDescription([
56         OpaqueFunction(function = launch_setup)
57     ])
```

◆ Main program entry:

```
79 if __name__ == '__main__':
80     # 创建一个LaunchDescription对象(create a LaunchDescription object)
81     ld = generate_launch_description()
82
83     ls = LaunchService()
84     ls.include_launch_description(ld)
85     ls.run()
```

① ld = generate_launch_description(): Generates a Launch description object.

② ls = LaunchService(): Creates a Launch service object.

③ ls.include_launch_description(ld): Adds the Launch description to the Launch service.

④ ls.run(): Starts the Launch service, executing all the defined Launch actions.

5.2 Analysis of Python Files

The source code is located at:

`/home/ubuntu/ros2_ws/src/large_models/large_models/vllm_track.py`

◆ Feature Package Import

```
6 import os
7 import cv2
8 import json
9 import time
10 import math
11 import queue
12 import rclpy
13 import threading
14 import numpy as np
15 import sdk.fps as fps
16 import message_filters
17 from sdk import common
18 from cv_bridge import CvBridge
19 from rclpy.node import Node
20 from sensor_msgs.msg import Image
21 from geometry_msgs.msg import Twist
22 from std_msgs.msg import String, Float32, Bool
23 from std_srvs.srv import Trigger, SetBool, Empty
24 from rcl_interfaces.msg import SetParametersResult
25 from rclpy.executors import MultiThreadedExecutor
26 from rclpy.callback_groups import ReentrantCallbackGroup
27
28 from speech import speech
29 from large_models.config import *
30 from large_models_msgs.srv import SetString, SetModel
31 from large_models.track_anything import ObjectTracker
```

- ① **cv2**: Used for image processing and display with OpenCV.
- ② **json**: Used for handling JSON-formatted data.
- ③ **.sdk.fps as fps**: Used to calculate and display the frame rate (FPS).
- ④ **speech**: Modules related to large model voice interaction.
- ⑤ **large_models_msgs.srv**: Custom service type for large models.
- ⑥ **large_models.config**: Configuration file for large models.

◆ **PROMPT Character String**

A prompt string (PROMPT) is defined to guide the processing of user commands and image data, specifying the rules and output format for the recognition task.

◆ **VLLMTrack Class**

```

48 display_size = [int(640*6/4), int(480*6/4)]
49
50 class VLLMTrack(Node):
51     def __init__(self, name):
52         rclpy.init()
53         super().__init__(name)
54         self.machine_type = os.environ['MACHINE_TYPE']
55         self.fps = fps.FPS() # 帧率统计器 (frame rate counter)
56         self.image_queue = queue.Queue(maxsize=2)
57         self.vllm_result = ''
58         # self.vllm_result = '''json{"object": "红色方块", "xyxy": [521, 508, 637, 683]}'''
59         self.running = True
60         self.data = []
61         self.start_track = False
62         self.bridge = CvBridge()
63         #cv2.namedWindow('image', 0)
64         #cv2.moveWindow('image', 1920 - display_size[0], 0)
65         #cv2.waitKey(10)
66         #os.system("wmctrl -r image -b add,above")
67
68         self.track = ObjectTracker(use_mouse=False, automatic=True, log=self.get_logger())
69         timer_cb_group = ReentrantCallbackGroup()
70         self.client = speech.OpenAIAPI(api_key, base_url)
71         self.mecanum_pub = self.create_publisher(Twist, '/controller/cmd_vel', 1) # 底盘控制
72         self.tts_text_pub = self.create_publisher(String, 'tts_node/tts_text', 1)
73         self.create_subscription(Bool, 'tts_node/play_finish', self.play_audio_finish_callback, 1, callback_group=timer_cb_group)
74         self.create_subscription(String, 'agent_process/result', self.vllm_result_callback, 1)
75         self.create_subscription(Bool, 'vocal_detect/wakeup', self.wakeup_callback, 1)
76
77         self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wakeup')
78         self.awake_client.wait_for_service()
79         self.set_model_client = self.create_client(SetModel, 'agent_process/set_model')
80         self.set_model_client.wait_for_service()
81         self.set_prompt_client = self.create_client(SetString, 'agent_process/set_prompt')
82         self.set_prompt_client.wait_for_service()
83
84         image_sub = message_filters.Subscriber(self, Image, 'ascamera/camera_publisher/rgb0/image')
85         depth_sub = message_filters.Subscriber(self, Image, 'ascamera/camera_publisher/depth0/image_raw')
86

```

① self.vllm_result is used to store the processed results. self.running: A boolean variable used to control the program's running state. self.data: An empty list used to store data. self.start_track: A boolean variable used to control the start or stop of target tracking.

② self.track = ObjectTracker(): Instantiates an object tracker.

③ self.tts_text_pub: Creates a publisher for publishing text to the text-to-speech synthesis node. self.track.update_pid(): Passes the updated PID parameters to the object tracker (self.track) to adjust the tracker's PID controller.

④ self.pid_params: PID controller parameters used in the control logic of the object tracker.

◆ On_parameter_update Method

```

113 def on_parameter_update(self, params):
114     """参数更新回调"""
115     for param in params:
116         if param.name in self.pid_params.keys():
117             self.pid_params[param.name] = param.value
118     # self.get_logger().info(f'PID parameters updated: {self.pid_params}')
119     # 更新 PID 参数
120     self.track.update_pid([self.pid_params['kp1'], self.pid_params['ki1'], self.pid_params['kd1']],
121                          [self.pid_params['kp2'], self.pid_params['ki2'], self.pid_params['kd2']])
122
123     return SetParametersResult(successful=True)

```

- ① self.pid_params: A dictionary used to store PID parameters.
- ② self.track.update_pid: Updates the PID parameters for the target tracking module.

◆ create_update_callback Method

```

125 def create_update_callback(self, param_name):
126     """生成动态更新回调"""
127     def update_param(msg):
128         new_value = msg.data
129         self.pid_params[param_name] = new_value
130         self.set_parameters([Parameter(param_name, Parameter.Type.DOUBLE, new_value)])
131         self.get_logger().info(f'Updated {param_name}: {new_value}')
132         # 更新 PID 参数
133     return update_param
134

```

Generates a dynamic update callback function for each PID parameter.

◆ get_node_state Method

```

136 def get_node_state(self, request, response):
137     return response

```

return response: Returns the response object.

◆ init_process Method

```

139 def init_process(self):
140     self.timer.cancel()
141
142     msg = SetModel.Request()
143     msg.model_type = 'vllm'
144     msg.model = stepfun_vllm_model
145     msg.api_key = stepfun_api_key
146     msg.base_url = stepfun_base_url
147     self.send_request(self.set_model_client, msg)
148
149     msg = SetString.Request()
150     msg.data = PROMPT
151     self.send_request(self.set_prompt_client, msg)
152
153     self.mecanum_pub.publish(Twist())
154     time.sleep(1.8)
155     speech.play_audio(start_audio_path)
156     threading.Thread(target=self.process, daemon=True).start()
157     self.create_service(Empty, '~/init_finish', self.get_node_state)
158     self.get_logger().info('\033[1;32m%s\033[0m' % 'start')

```

- ① SetModel.Request: Creates a request message to set the model.
- ② SetString.Request: Defines the request message for setting the

prompt string.

- ③ set_pose_target: Sets the robot's initial pose.
- ④ Self.send_request: Sends a service request.
- ⑤ speech.play_audio: Plays an audio clip.
- ⑥ threading.Thread: Starts a new thread for image processing and tracking.

◆ Send_request Method

```
160     def send_request(self, client, msg):
161         future = client.call_async(msg)
162         while rclpy.ok():
163             if future.done() and future.result():
164                 return future.result()
```

- ① Client.call_async(msg): Asynchronously calls a service.
- ② future.done() and future.result(): Check if the service call has completed and retrieve the result.

◆ vllm_result_callback method:

```
171     def vllm_result_callback(self, msg):
172         self.vllm_result = msg.data
```

Receives results from the agent_process/result topic and stores them in self.vllm_result.

◆ play_audio_finish_callback method

```
174     def play_audio_finish_callback(self, msg):
175         if msg.data:
176             msg = SetBool.Request()
177             msg.data = True
178             self.send_request(self.awake_client, msg)
```

Sends a wake-up signal once audio playback is completed.

◆ process Method

```

215 def process(self):
216     box = ''
217
218     while self.running:
219         image, depth_image = self.image_queue.get(block=True)
220
221         if self.vllm_result:
222             try:
223                 self.vllm_result = json.loads(self.vllm_result)
224                 box = self.vllm_result['xyxy']
225                 if box:
226                     if os.environ["ASR_LANGUAGE"] == 'Chinese':
227                         box = self.client.data_process(box, 640, 480)
228                     else:
229                         box = [int(box[0] * 640), int(box[1] * 480), int(box[2] * 640), int(box[3] * 480)]
230                 box = [box[0], box[1], box[2] - box[0], box[3] - box[1]]
231                 self.track.set_track_target(box, image)
232                 self.start_track = True
233                 speech.play_audio(start_track_audio_path, block=False)
234             except (ValueError, TypeError) as e:
235                 self.start_track = False
236                 msg = String()
237                 msg.data = self.vllm_result
238                 self.tts_text_pub.publish(msg)
239                 speech.play_audio(track_fail_audio_path, block=False)
240                 self.get_logger().info(e)
241             self.vllm_result = ''
242             msg = SetBool.Request()
243             msg.data = True
244             self.send_request(self.awake_client, msg)
245         if self.start_track:
246             self.data = self.track.track(image, depth_image)
247             image = self.data[-1]
248             twist = Twist()
249             twist.linear.x, twist.angular.z = self.data[0], self.data[1]
250             # self.get_logger().info('twist.linear.x:{}'.format(twist.linear.x))
251             # self.get_logger().info('twist.angular.z:{}'.format(twist.angular.z))
252
253             if self.machine_type == 'MentorPi_Acker':
254                 steering_angle = common.set_range(twist.angular.z, -math.radians(322/2000*180), math.radians(322/2000*180))
255                 if steering_angle != 0:
256                     R = 0.145/math.tan(steering_angle)
257                     twist.angular.z = twist.linear.x/R
258

```

① Image data is retrieved from the image_queue.get queue. The recognition results are then received via the vllm_result subscriber. These results are parsed using json.load and processed accordingly. Finally, the track.set_track_target function is called to set the tracking target.

② speech.play_audio(start_track_audio_path, block=False): Plays an audio file to indicate that target tracking has started.

③ self.send_request(self.awake_client, msg): Sends a service request via awake_client to enable voice wake-up functionality.

◆ Multi_callback Method

Converts received RGB image messages into NumPy arrays and stores them in the queue.

◆ Main Method

```
255 def main():
256     node = VLLMTrack('vllm_track')
257     executor = MultiThreadedExecutor()
258     executor.add_node(node)
259     executor.spin()
260     node.destroy_node()
```

Creates an instance of the VLLMTrack node. Uses a multithreaded executor to handle node tasks. Calls executor.spin() to start processing ROS events.

Destroys the node when finished by calling node.destroy_node().